

Commandes et suggestions

Cette mise à jour ajoute:

- Un bouton **Commande** dans l'onglet **Ajouter** pour générer des suggestions automatiques (approche heuristique simple — **qHash** sur le nom du fournisseur) et proposer d'enregistrer ces suggestions dans la BD si la table **COMMANDES** existe.
- Un bouton **Statistic** qui affiche des statistiques de base (nombre total et répartition par initiale).
- Un bouton **Search** (**modifsearch**) dans l'onglet **Modifier** qui recherche un fournisseur par ID et remplit les champs **Nom** et **Téléphone** si trouvé.
- Un bouton **Tri par name** pour trier la liste locale des fournisseurs par **nom**.

SQL pour ajouter la table **COMMANDES**

Voici un exemple SQL pour créer une table **COMMANDES** dans Oracle. Adaptez les types selon vos besoins :

```
CREATE TABLE COMMANDES (  
  ID_COMMANDE NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,  
  ID_FOURNISSEUR NUMBER NOT NULL,  
  PRODUIT VARCHAR2(200),  
  QUANTITE NUMBER,  
  DATE_SUGGESTION DATE  
);  
  
-- Index utile  
CREATE INDEX IDX_COMMANDES_IDFOURN ON COMMANDES(ID_FOURNISSEUR);
```

Après avoir créé cette table, le bouton **Commande** pourra insérer les suggestions. Si la table n'existe pas, l'application propose uniquement d'afficher les suggestions.

Notes de fonctionnement

- Les suggestions utilisent une heuristique simple: le **qHash** du nom du fournisseur, mappé à une liste de produits et à une quantité.
- Les suggestions utilisent maintenant une logique plus intelligente et aléatoire :
 - L'algorithme va récupérer l'historique des commandes (table **COMMANDES**) si elle existe.
 - Il calcule des pondérations globales par produit (produits les plus commandés) et pondère plus fortement les produits propres au fournisseur.
 - Il choisit aléatoirement un produit selon ces pondérations (mais différent à chaque exécution grâce à **QRandomGenerator**).
 - La quantité est soit basée sur la moyenne historique pour le produit, soit choisie aléatoirement si aucun historique n'existe.
 - Les suggestions sont enregistrées en mémoire lors de la génération afin que, si vous acceptez d'enregistrer dans la BD, les mêmes suggestions affichées soient celles qui seront insérées.

- Pour une vraie fonctionnalité "AI", il faudrait entraîner un modèle (ou appeler une API) en fonction de l'historique des commandes et des fournisseurs; ici on donne un comportement inoffensif mais prédictif basé sur le nom.
- Ces règles offrent un comportement plus aléatoire et plus pertinent. Si vous voulez des suggestions réellement intelligentes, je peux intégrer un modèle ML ou appeler un service externe (ex. un petit modèle auto-hébergé ou GPT-like) qui utiliserait l'historique complet.
- Pour la modification via ID: si le fournisseur n'est pas en mémoire, l'application interroge la base (requête par ID) et complète le formulaire.

Commandes de build (Windows PowerShell)

Ouvrez PowerShell dans le dossier du projet et exécutez :

```
qmake interface.pro  
mingw32-make
```

Puis lancez l'exécutable `interface.exe`.

Si vous voulez, je peux ajouter un écran dédié pour afficher et gérer (CRUD) les commandes sauvegardées, ou créer un petit panneau pour paramétrer les règles de suggestion. Dites-moi ce que vous préférez.